



INTERNET OF THINGS (24BECSE307) LAB MANUAL



DEPT OF CSE
SAPTHAGIRI NPS UNIVERSITY

List of Experiments

Introduction to IoT Development Boards: Arduino IDE Software, ESP32, and Raspberry Pi(micro python)

1. Develop a program to illustrate the working of LED with a push button.
2. Write a program to blink an LED with different times and duration using the concept of user defined function.
3. Develop a program to rotate servo motor both in clockwise and anticlockwise direction.
4. Develop a program to simulate the working of LCD and print the given text on LCD.
5. Develop a program to calculate the distance of an object using ultrasonic sensor.
6. Develop a program to interface temperature sensor to read the room temperature, humidity and heat index and print the readings on the serial monitor.
7. Write a program to demonstrate how ESP32/ Raspberry Pi can be used as an HTTP client and HTTP server to control and monitor the status of an LED
8. Write a program demonstrate how ESP32/ Raspberry Pi can be used as HTTP Webserver and get commands from the client (mobile/Laptop) directly.
9. Develop a IOT App using cloud (optional)
10. Develop a IOT App Menus using Template(optional)
11. Write a program to demonstrate how ESP32/ Raspberry Pi can be used to log sensor data into thingspeak /Blynk IoT cloud.

Experiment 1

A program to illustrate the working of LED with a push button.

Introduction:

Push-button is a very simple mechanism which is used to control electronic signal either by blocking it or allowing it to pass. This happens when mechanical pressure is applied to connect two points of the switch together. Push buttons or switches connect two points in a circuit when pressed. When the push-button is released, there is no connection between the two legs of the push-button. Here it turns on the built-in LED on pin 11 when the button is pressed. The LED stays ON as long as the button is being pressed.

LED



Pin definition	
Long pin	+5V
Short pin	GND

Push Button

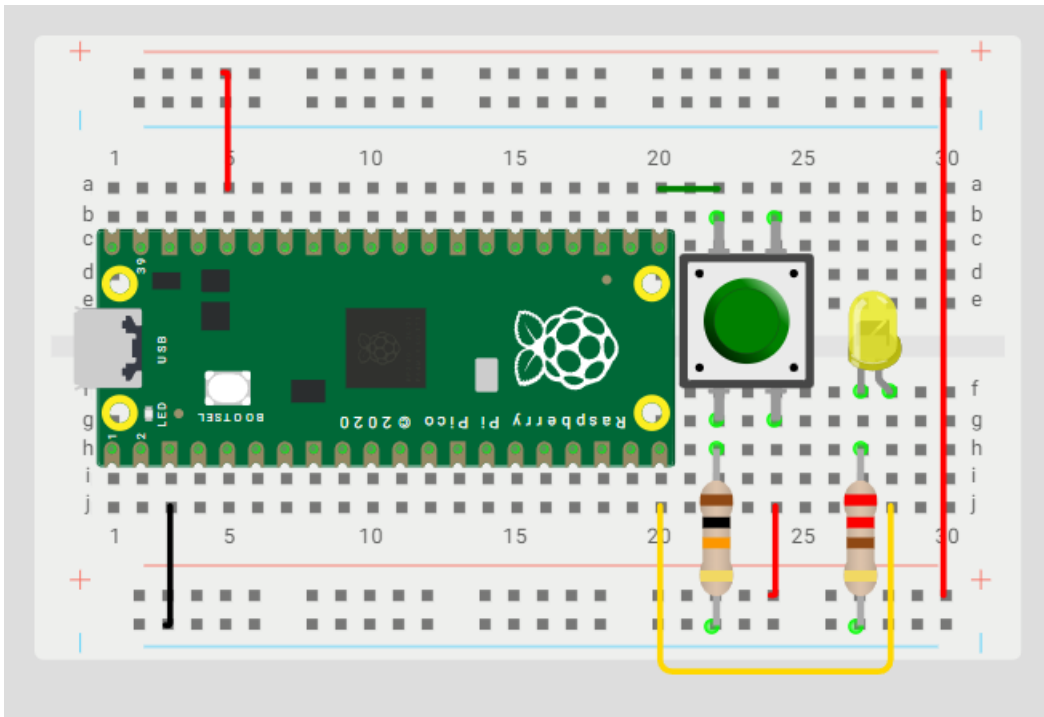


Hardware Required:

Component Name	Quantity
Raspberry Pi	1
LED	1
Push Button	1
220Ω resistor	1
10KΩ resistor	1
USB Cable	1
Breadboard	1
Jumper wires	Several



Connection Diagram:



Steps of working

1. Insert the push button into your breadboard and connect it to the digital pin GPIO Pin 16 which act as INPUT.
2. Insert the LED into the breadboard. Attach the negative leg to GPIO pin 15 of the Raspberry Pi, and the positive leg via the 220-ohm resistor to GND. The pin 15 is taken as OUTPUT.
3. Upload the code as given below.
4. Press the push-button to control the ON state of LED. This sketch works by setting pin 16 as for the push button as INPUT and pin 15 as an OUTPUT to power the LED. The initial state of the button is set to OFF. After that the code run a loop that continually reads the state from the pushbutton and sends that value as voltage to the LED. The LED will be ON accordingly.



Code

```
# modules
from machine import Pin
from time import sleep

LED = Pin(15, Pin.OUT)      # creating LED object, setting it as OUT
BUTTON = Pin(16, Pin.IN)    # creating Push Button object, setting it as IN

# continuously read the signal from the push button while the board has power
while True:
    # if the signal from the button is HIGH (button is pressed), turn LED on
    if BUTTON.value() == 1:
        LED.on()
        sleep(0.1)

    # otherwise, turn the LED off
    else:
        LED.off()
```

Observation Table:

Sl No.	Push button State	LED State
1		
2		

Experiment 2

A program to blink an LED with different times and duration using the concept of user defined function.

Introduction:

- The GPIO pin connected to the LED is initialized as an output using the Pin object.
- A user-defined function blink_led is written to make the LED blink a specified number of times, with customizable ON and OFF durations.
- The blinking action is managed using a for loop, which repeatedly turns the LED ON and OFF with delays in between.

LED

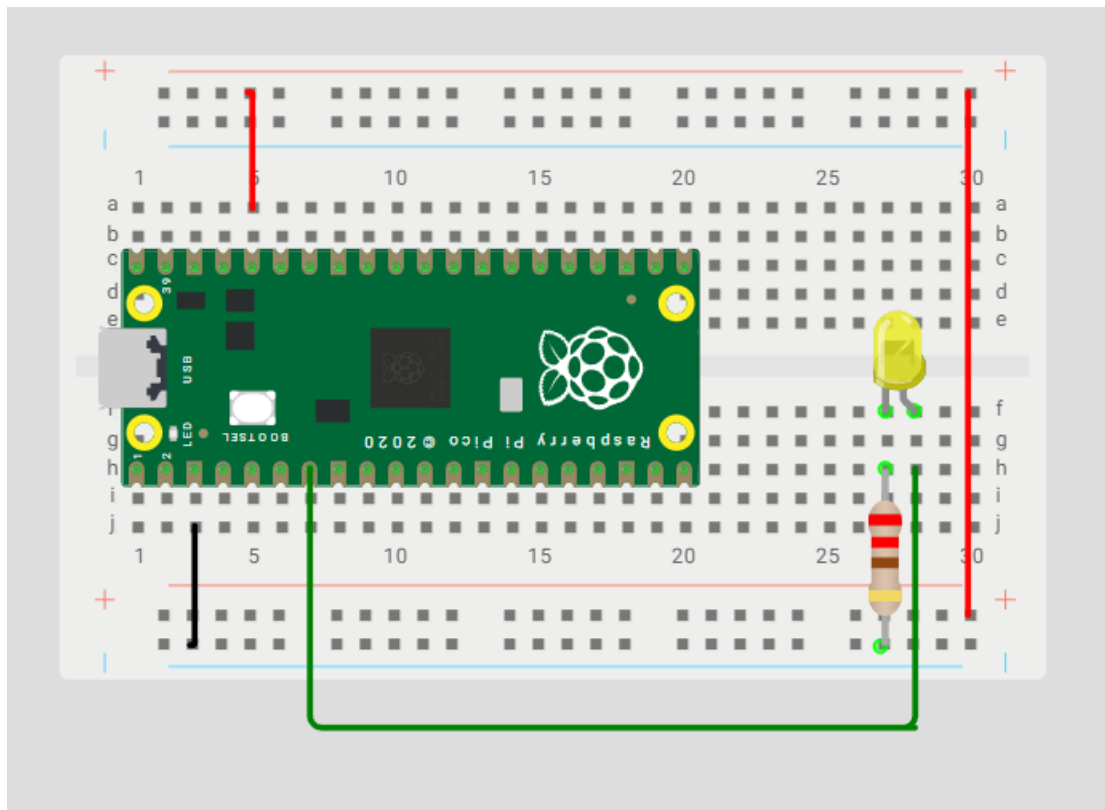


Pin definition	
Long pin	+5V
Short pin	GND

Hardware Required:

Component Name	Quantity
Raspberry Pi	1
LED	1
220Ω resistor	1
USB Cable	1
Breadboard	1
Jumper wires	Several

Connection Diagram:



Steps of working

1. Insert the LED into the breadboard. Attach the negative leg to GPIO pin 5 of the Raspberry Pi, and the positive leg via the 220-ohm resistor to GND. The pin 5 is taken as OUTPUT.
2. Upload the code as given below.
3. The LED will be ON according to the input parameters.

Code

```
from machine import Pin
from utime import sleep

# Initialize the LED pin
led = Pin(5, Pin.OUT) # Replace 5 with the GPIO pin number connected to the LED

# User-defined function to blink the LED
def blink_led(times, duration_on, duration_off):
    """
    Blink the LED a specified number of times with custom on/off durations.

    :param times: Number of times to blink the LED
    """
```



SAPTHAGIRI NPS
UNIVERSITY

UNMATCHED EXCELLENCE, UNLIMITED POTENTIAL

```
:param duration_on: Duration (in seconds) the LED stays ON
:param duration_off: Duration (in seconds) the LED stays OFF
"""
```

```
for _ in range(times):
    led.value(1) # Turn LED ON
    sleep(duration_on)
    led.value(0) # Turn LED OFF
    sleep(duration_off)
```

```
# Example usage of the function
```

```
while True:
```

```
    blink_led(3, 0.5, 0.5) # Blink 3 times with 0.5s ON and 0.5s OFF
    sleep(2) # Pause before the next sequence
    blink_led(5, 0.2, 0.8) # Blink 5 times with 0.2s ON and 0.8s OFF
    sleep(3) # Pause before the next sequence
```


Experiment 3

A program to rotate servo motor both in clockwise and anticlockwise direction.

Introduction:

A servo motor is a compact and precise actuator widely used in robotics, automation, and control systems. It is capable of rotating its shaft to a specified angular position, typically between 0° and 180° . Controlling a servo motor requires a Pulse Width Modulated (PWM) signal, where the duty cycle of the pulse determines the angle of rotation.

In this experiment, we use a Raspberry Pi Pico, a microcontroller board based on the RP2040 chip, to control a servo motor in both clockwise and anticlockwise directions. The Pico generates PWM signals on its GPIO pins, which makes it well-suited for servo applications. By writing a MicroPython program, we can sweep the servo from 0° to 180° and back, thereby achieving bidirectional rotation.

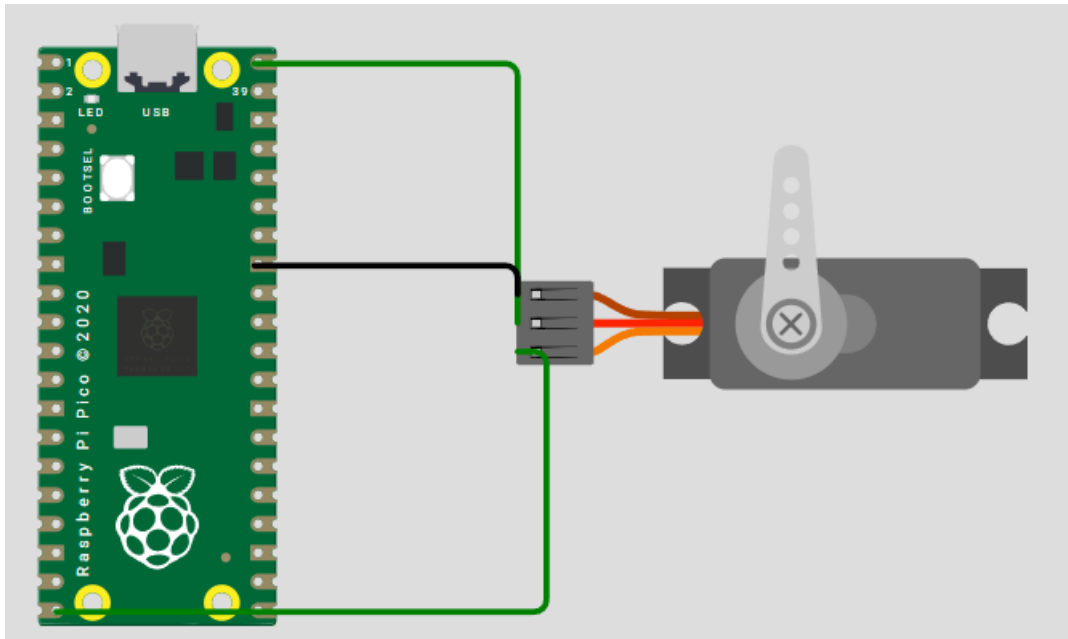


Hardware Required:

Component Name	Quantity
Raspberry Pi	1
Servomotor	1
Jumper wires	Several



Connection Diagram:



Steps of working

1. Connect the **signal (PWM)** wire of the servo motor to **GPIO15** of Raspberry Pi Pico.
2. Connect the **VCC pin** of the servo motor to the **VBUS (5V)** pin of the Pico.
3. Connect the **GND pin** of the servo motor to the **GND** pin of the Pico.
4. Upload the code as given below.
5. Run the code and observe the rotation of servomotor.

Code

```
from machine import Pin, PWM
from time import sleep
```

```
# Initialize servo on GPIO pin 15
servo = PWM(Pin(15))
servo.freq(50) # Standard servo frequency: 50 Hz
```

```
# Function to set angle
def set_angle(angle):
    duty = int(((angle / 180) * 5000) + 1000) # Convert angle to duty cycle (1000–8000 range)
    servo.duty_u16(duty)
```

```
while True:
    # Clockwise rotation (0° → 180°)
    for angle in range(0, 181, 10):
        set_angle(angle)
        sleep(0.05)
```



SAPTHAGIRI NPS
UNIVERSITY
UNMATCHED EXCELLENCE, UNLIMITED POTENTIAL

sleep(1)

Anticlockwise rotation ($180^\circ \rightarrow 0^\circ$)

for angle in range(180, -1, -10):

 set_angle(angle)

 sleep(0.05)

sleep(1)



Experiment 4

Develop a program to simulate the working of LCD and print the given text on LCD.

Introduction:

A Liquid Crystal Display (LCD) is a flat-panel display that uses liquid crystals to modulate light. The 16x2 LCD module is widely used in embedded systems because it can display 16 characters per line on 2 rows, making it ideal for projects that require text-based information output.

Unlike LEDs or 7-segment displays, the LCD allows the display of letters, numbers, and custom characters. By interfacing the LCD with a microcontroller such as the Arduino Uno or Raspberry Pi Pico, we can print messages and simulate the working of the LCD. The microcontroller sends data and control signals to the LCD either in parallel mode (4-bit/8-bit) or via I2C communication, depending on the hardware.

- ☐ **4-bit mode** is used (only D4–D7 are connected).
- ☐ **RS pin** selects whether you're sending **command** or **data**.
- ☐ **E pin** is the "enable strobe" that tells LCD when to read data.
- ☐ **VCC** and **GND** power the LCD module (5V operation).

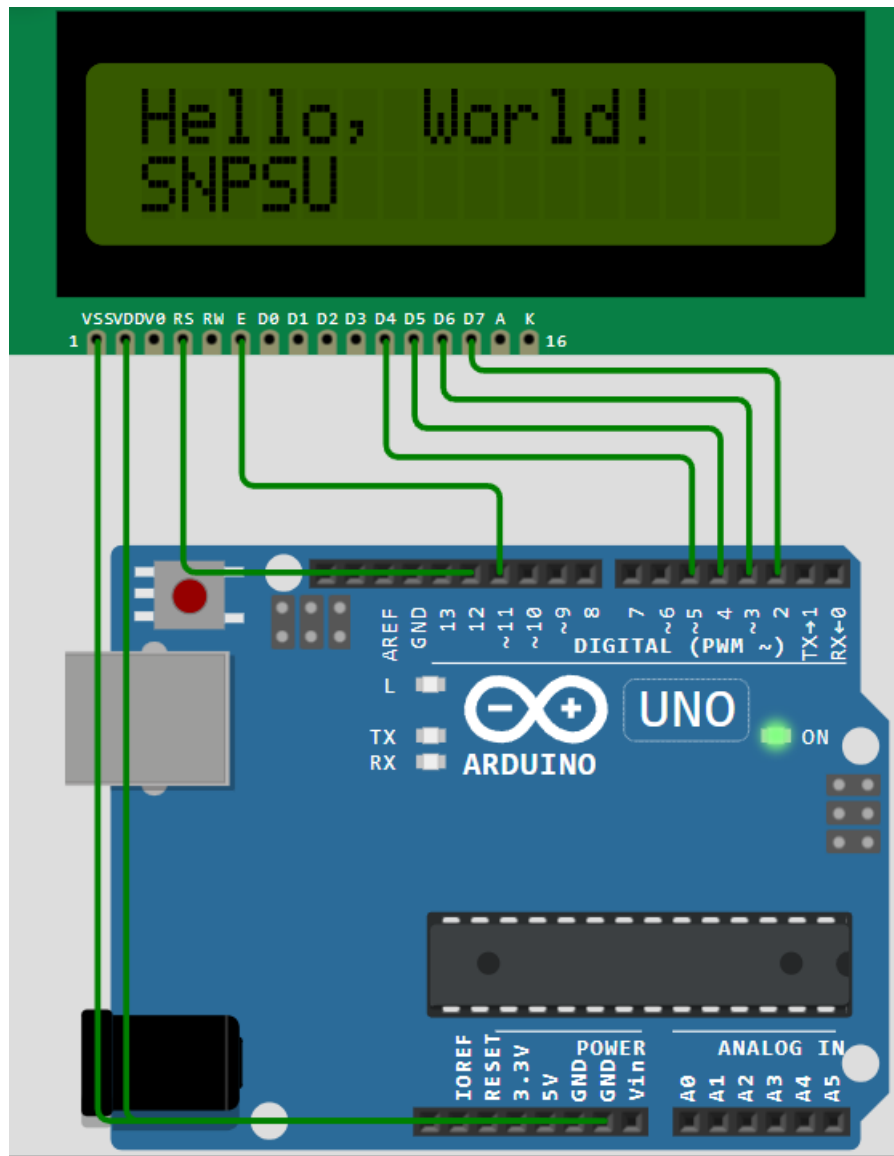
This experiment demonstrates how to simulate the working of an LCD and display user-defined text.

Hardware Required:

Component Name	Quantity
Aurdino Uno	1
LCD (16x2)	1
Jumper wires	Several



Connection Diagram:



Steps of working

1. Connect the LCD module to the microcontroller (Arduino Uno) as per the pin configuration.

LCD Pin	Arduino UNO Pin	Function
RS	D12	Register Select (data/command select)
E	D11	Enable signal
D4	D5	Data bit 4
D5	D4	Data bit 5



D6	D3	Data bit 6
D7	D2	Data bit 7
VDD	5V	Power supply (+5V)
VSS	GND	Ground (0V)

2. Upload the code as given below. Save with .ino extension.
3. Run the code and observe the LCD display "Hello, World!" on the first line and "SNPSU" on the second line..

Code

```
#include <LiquidCrystal.h>
```

```
// LCD pin mapping: RS=12, E=11, D4=5, D5=4, D6=3, D7=2  
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```
void setup() {  
  lcd.begin(16, 2);      // Initialize LCD as 16x2  
  lcd.print("Hello, World!"); // Print text on first line  
}
```

```
void loop() {  
  lcd.setCursor(0, 1);    // Move to second line  
  lcd.print("SNPSU"); // Print text  
}
```



Experiment 5

A program to calculate the distance of an object using ultrasonic sensor.

Introduction:

The HC-SR04 is a widely used ultrasonic distance sensor designed to measure the distance between the sensor and an object without physical contact. It works on the principle of ultrasonic sound waves and the time-of-flight method. The sensor transmits high-frequency sound waves (40 kHz), which travel through the air. When these waves hit an object, they are reflected back toward the sensor. By measuring the time taken for the echo to return, the distance to the object can be calculated. The working principle is as follows:

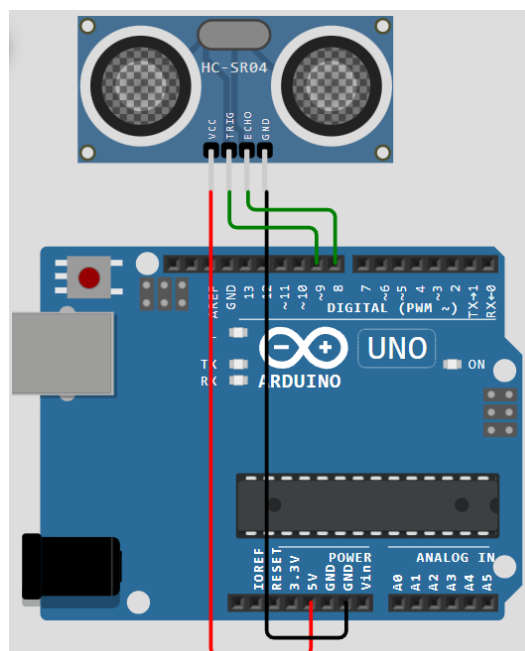
1. Arduino sends 10 μ s trigger pulse to TRIG.
2. Sensor transmits 8 ultrasonic waves (40 kHz).
3. Waves hit an obstacle and reflect back.
4. Sensor sets ECHO pin HIGH for the duration of travel.
5. Arduino measures the HIGH pulse width.
6. Distance is calculated using the formula:

$$\text{Distance (cm)} = \frac{\text{Time } (\mu\text{s}) \times 0.03432}{2}$$

Hardware Required:

Component Name	Quantity
Aurdino Uno	1
HC-SR04	1
Jumper wires	Several

Connection Diagram:





Steps of working

1. Connect HC-SR04 to Aurdino board as follows:

Ultrasonic Sensor Pin	Arduino UNO Pin	Purpose
VCC	5V	Powers the ultrasonic sensor
GND	GND	Common ground
TRIG	D9	Arduino sends a 10µs pulse to start measurement
ECHO	D8	Sensor outputs HIGH pulse proportional to distance

2. Upload the code as given below.
3. Run the code and observe the distance in serial monitor.
4. To change the distance in Wokwi, click on the **Ultrasonic Sensor** to see a **distance slider** (e.g., 10 cm → 400 cm). Move the slider . Then the value in Serial Monitor changes in real time

Code

```
#define TRIG_PIN 9
#define ECHO_PIN 8

void setup() {
  Serial.begin(9600);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
}

void loop() {
  // Send a 10µs pulse
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // Read the echo pulse duration
  long duration = pulseIn(ECHO_PIN, HIGH);

  // Convert time to distance (cm)
  float distance = duration * 0.0343 / 2;
```




SAPTHAGIRI NPS
UNIVERSITY
UNMATCHED EXCELLENCE, UNLIMITED POTENTIAL

```
Serial.print("Distance: ");  
Serial.print(distance);  
Serial.println(" cm");
```

```
delay(500);  
}
```



Experiment 6

A program to interface temperature sensor to read the room temperature, humidity and heat index and print the readings on the serial monitor.

Introduction:

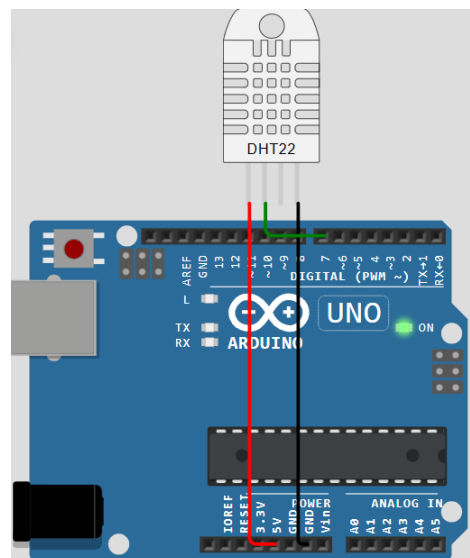
The DHT22 (also known as AM2302) is a low-cost, digital-output sensor used to measure temperature and relative humidity. It combines a capacitive humidity sensor and a thermistor in a single package and provides calibrated digital output via a single data line, making it simple to interface with microcontrollers like Arduino, Raspberry Pi, and ESP32. The working principle is as follows:

- Arduino sends a start signal to the DHT22 through the DATA pin.
- The DHT22 responds with a digital signal containing temperature and humidity values.
- Arduino decodes this signal using the DHT library.
- The code also computes the heat index using temperature + humidity formula.
- Finally, all values are printed on the Serial Monitor.

Hardware Required:

Component Name	Quantity
Aurdino UNO	1
DHT22	1
Jumper wires	Several

Connection Diagram:





Steps of working

1. Connect DHT22 sensor to Aurdino Board as follows:

DHT22 Pin	Arduino UNO Pin	Purpose
VCC	5V	Provides operating voltage to the sensor
GND	GND	Common ground
SDA	D7	Arduino reads temperature & humidity data

2. Upload the code as given below.
3. Run the Simulation to see the readings in serial monitor
4. Click on the DHT22 sensor in the workspace. Adjust the temperature and humidity sliders provided in Wokwi. Observe the updated readings (Temperature, Humidity, Heat Index) on the Serial Monitor.

Code

```
#include "DHT.h"

#define DHTPIN 7    // Pin connected to the DHT sensor
#define DHTTYPE DHT22 // Can be DHT11 or DHT22

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  Serial.println("DHT22 Sensor Reading...");
  dht.begin();
}

void loop() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();    // Celsius
  float heatIndex = dht.computeHeatIndex(temperature, humidity, false);

  // Check if any reading failed
  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

  Serial.print("Temperature: ");
  Serial.print(temperature);
```



SAPTHAGIRI NPS
UNIVERSITY

UNMATCHED EXCELLENCE, UNLIMITED POTENTIAL

```
Serial.println(" °C");
```

```
Serial.print("Humidity: ");  
Serial.print(humidity);  
Serial.println(" %");
```

```
Serial.print("Heat Index: ");  
Serial.print(heatIndex);  
Serial.println(" °C");
```

```
Serial.println("-----");
```

```
delay(2000); // Wait 2 seconds before next reading  
}
```